

ParentPal

An AI parenting companion. Parents log short "moments" about their child, pick a goal or two, get small scripted "wins" to try, and ask questions in a chat. The chat answers from curated, sourced content and from what the parent has logged.

This is a portfolio MVP. It covers one working path through the app from start to finish, and the docs try to be honest about its limits.

Not medical advice. All guidance is general, written from public sources, and has not been checked by a clinician. Every AI-generated surface in the app says so.

Features

Area	What it does
Onboarding	Guest-first: child (nickname, sex, birth date picker, optional second child) → intro slides → pick 1–2 goals → parent details → fake paywall. Closing the paywall drops you into the free tier.
Home	Personalized goals, category filter chips, goal detail pages with 4–5 wins. Win 1 is free and the rest unlock with a (fake) subscription.
Story	Log moments in free text. An LLM tags each one as trigger / behavior / outcome. After every 3 moments it writes a pattern insight that links back to the moments it came from.
Ask	Streaming chat (SSE). Answers draw on the child's age, active goals and recent moments, plus retrieved content shown as citations. Vague questions get one clarifying question with tap-to-answer options. Replies can be rated 👍 / 👎, copied, shared and bookmarked.
Notifications	On-device reminders that serve the try → log → pattern loop: a daily idea sent when it's usable (sleep ideas before bedtime), a "How did it go?" check-in the morning after tapping I'll try this on a win (opens the moment log, filled in), and one nudge after 3 days without a moment. At most one a day, never at night, each type switchable in Settings. Tips also collect in an in-app inbox. See How notifications work .
Progress	Every I'll try this is saved as a try. The parent taps how it went (Helped / A bit / Not yet) on the goal screen, or from the next-morning check-in. Each win shows "You tried this 4× · helped 3" and a next step ("This is working, keep going" or "Not helping yet. Try win 2?"). Goals show recent outcomes as dots and, after a week, week 1 against this week. Caregivers' "It worked / It was tough" counts too. See Progress tracker .
Circles	Anonymous groups by goal, with posts tagged by the child's age band. Each parent gets a different nickname in each circle. Three post types: question, what worked (tied to a win, with an outcome) and sharing. Questions get a cited "ParentPal guide" reply straight away. "What worked" reports add up to a "Parents like you: 5 of 7 said it helped" line on the win. Every post and reply is moderated before anyone sees it. See How Circles work .

Area	What it does
Family playbook	Share the wins you're trying, with the exact words to say, with grandparents, the other parent, a nanny or a teacher. Each person gets a private link that needs no app or login. They can send back "It worked" or "It was tough" plus a note, which lands in the child's Story as "Logged by Nani" and counts towards patterns. See Feature brief: Family Playbook .
Two children	A switcher (shown only with two children) picks which child Home, Story, Ask and the goal screens are about. Moments, patterns, progress and chat answers are that child's; caregiver links and Circles posts say which child they're for. See Two children .
Profile	Family profile you can edit (names, role, birth dates, add or remove a child), sign in / create account (upgrades the guest account), forgot password (emailed 6-digit code), sign out, bookmarks, manage subscription (stub), refer friends (stub), hard account deletion.
Safety	A rule-based red-flag check runs on every chat message and moment <i>before</i> any LLM call. Covered: medical emergencies, abuse, self-harm, developmental concerns. On a match the app skips the advice and shows fixed safety guidance instead.

What's new: features built on top of the original idea

The first version of ParentPal follows the original product: goals and wins, an AI chat, a moment journal with patterns, and notifications. Larger features were then added that the original doesn't have, plus several smaller improvements. Each is described below in full: what it is, every screen and control, the rules behind it, and how it was tested. Product reasoning for each one is in its feature brief further down.

	Feature	One line
1	Circles	Anonymous groups of parents by goal and child's age, with instant expert-grounded answers and AI moderation.
2	Family Playbook	Share the plan and the exact words with grandparents, a nanny or a teacher on a no-app link; their notes flow into Story.
3	Smaller improvements	"Ask other parents" from chat, community stats on wins, reply notices in the inbox, coloured Story pebbles, and fixes.
4	Quality work	26 new automated tests, live-model checks, and three new entries in the AI mistakes log.
5	Bug-fix pass	A review of the whole app: paid content no longer leaks through chat, no more 500s from odd input, and screens that used to fail silently now say so.
6	Progress tracker	Each try of a win gets an outcome, so parents see what's working, get a next step, and see week 1 against this week.
7	Two children	A child switcher on Home, Story and Ask; moments, patterns, progress, chat, caregiver links and Circles follow the chosen child.

1. Circles: anonymous parent community

What it is. One circle per goal (Tantrums, Sleep, Picky eating, Screen time, Focus, Potty training, Keeping busy). Every post carries the child's age band (Under 1, 1 to 2, 2 to 3, 3 to 5, 5 and up), so parents read

about children the same age as theirs.

Screens and what's on them

Screen	What the parent sees and can do
Circles tab (new tab, between Ask and Notifications)	"Your circles" (from the goals they picked) first, then "More circles". Each circle shows its illustration and activity ("4 posts this week", or "Quiet this week. Start the conversation"). Guests see a "You're reading as a guest" card with Create free account . A "How Circles work" card lists the house rules.
Circle feed (circle/[goal])	A coloured header with New post . Age-band chips, with the parent's own band preselected and marked "(yours)", plus "All ages". Post cards show the author's nickname and label ("Gentle Robin · Mom of a 2-year-old"), time ago, the kind badge (Question / What worked / Sharing), the win and outcome for "what worked" posts, the text, reactions and reply count. Load more pages through older posts. If the chosen age is quiet, a "From nearby ages" section shows posts from the neighbouring bands.
Post (post/[id])	The full post. A pinned support card if it mentions a developmental worry. The ParentPal guide reply (green card, "From our expert-sourced guides, not another parent"), with numbered sources that open the goal. Then replies from parents, each with reactions and a ... menu. A reply box at the bottom; guests see "Create a free account to reply".
... menu	On your own post or reply: Delete (with confirmation). On someone else's: a bottom sheet to Report (Unkind or shaming / Risky medical advice / Shares personal info / Spam or selling) or Block this parent . Guide replies can be reported but not blocked.
New post (post/new , modal)	Circle picker (when opened from outside a circle), post type chips, and for "What worked": a win picker and outcome chips ("It helped" / "Helped a bit" / "Didn't help"). A text box with character counter (20–600), and a reminder: "Be kind. No names, numbers or links. Your child's name is replaced automatically. No medicine doses or diagnoses." Post anonymously . Afterwards it opens the post, or shows "Your post will appear once a moderator has checked it", or a support screen for a crisis.

The rules

- **Anonymity:**
- A nickname like "Gentle Robin" is generated per parent per circle, keyed with a server secret, so one parent can't be linked across circles.
- The label shows only "Mom/Dad/Parent of a 2-year-old".
- The API never returns another user's id, email, name or child's nickname.
- The author's own children's nicknames are replaced with "my child" (capitalised at the start of a sentence).
- **Who can do what:** guests can read everything. Posting, replying, reacting and reporting need a free account, which cuts spam. Blocking works for everyone.
- **Moderation**, in order, for every post and reply:
 1. Phone numbers, emails, links and @handles are rejected with a message naming what to remove.
 2. Nicknames are replaced.
 3. The safety rules run:
 - Emergencies, self-harm and abuse are **not published**; the author sees helplines.
 - A developmental worry is published with the "talk to your pediatrician" card pinned.
 4. A rule holds anything mentioning a medicine or dose for review.
 5. An LLM decides allow / review / block:

- **Review** (only the author sees it, marked "Pending review"): a diagnosis of someone else's child, or off-topic content.
- **Block** (rejected with a kind explanation): insults, shaming, selling, or "DM me".
- If the LLM is down, the item **waits for review** instead of going live.
- **Reports**: 3 reports from different parents hide an item until a moderator decides. Once a moderator approves an item, reports alone can't hide it again.
- **Blocking**: hides a parent's posts and replies in every circle, for the person who blocked them only.
- **Limits**: 5 posts and 30 replies per parent per day.
- **Guide reply**:
 - Every question that goes live gets one reply from ParentPal's guides, built with the same prompt and citation checks as chat, with no family details.
 - It only answers when the content library matches strongly and from **that circle's own goal**, and it is dropped if it can't cite a source.
 - Crisis and developmental-worry posts never get one.
- **"What worked" stats**: reports roll up per win (distinct parents, live posts only) into "Parents like you: 5 of 7 who tried this said it helped" on the win card, shown only once 3 or more parents have reported.
- **Notifications**: when someone replies (or the guide answers), the author gets one inbox notice per post per day, e.g. "2 new replies to your post, plus a guide answer". Tapping it opens the post.
- **Moderator tools**: GET /v1/admin/community/queue lists held and hidden items with the reasons and reports. POST /v1/admin/community/:type/:id with approve or remove decides.

Built with: 6 new tables (community_posts , community_replies , community_reactions , community_reports , community_blocks , community_notices), migration 0002_community.sql , services/community.ts , services/moderation.ts , routes/community.ts , and 4 new mobile screens plus components/PostCard.tsx .

2. Family Playbook: share the plan with every caregiver

What it is. The parent sends grandparents, the other parent, a nanny or a teacher a private link. It opens a simple page, with no app and no login, showing the wins the family is working on and the exact words to use. The caregiver can say how it went, and that note goes into the child's Story.

Screens and what's on them

Screen	What you see and can do
Home card	"Get the whole family on the same page. Send Aarav's plan to Nani, Dad or the nanny. No app needed." Opens the Family playbook screen.
Profile row	"Family playbook: share the plan with grandparents, nanny or teacher."
Win card (goal screen)	A Send to family button that adds that win to the playbook and opens the Family playbook screen.

Screen	What you see and can do
Family playbook screen (family)	A short explanation of why consistency matters. What you're sharing: chips for every win the parent has unlocked, with the shared ones selected (up to 4), and each shared win's "Say this" script in an apricot callout. Shared with: one row per person with name, relationship, "Opened 2h ago" or "Not opened yet", and notes count, plus share again and stop sharing buttons. Add someone: "What does your child call them?" (e.g. Nani, Papa, Aunty Meena), relationship chips (Grandparent, Other parent, Nanny, Teacher, Other), and Create link and share , which opens the phone's share sheet with a ready-to-send message.
Caregiver page (/p/<link> , in any phone browser)	"Hi Nani! Here's what we're trying with Aarav right now, shared by Priya (Mom)." One card per win: goal tag, title, What to do , Say this (large type), What to expect . How did it go with Aarav?: big It worked / It was tough buttons, an optional "What happened?" box, and Send note . Afterwards: "Thank you! Your note was sent", or for a red flag, helplines straight away. Styled with the app's colours, fonts and pebble logo.
Story	Caregiver notes appear like any moment, marked "Logged by Nani", tagged by the AI (before / what happened / how it ended), and count towards patterns.

The rules

- **Entitlement:** until the parent chooses, the playbook shares win 1 of each of their goals. Once they've chosen, it shares exactly their choice, which can be nothing. Only wins the parent has unlocked can be shared (win 1 free, the rest with a subscription), up to 4 at a time.
- **Links:**
 - Each link is a 32-byte random token and works until the parent stops sharing; it then shows "Link turned off".
 - Up to 5 people per family.
 - Notes already sent stay in Story after a link is turned off.
- **Privacy:** the page shows only the child's nickname, the parent's first name and role, and the shared wins. Never chat, other moments, patterns or email.
- **Page security:** the page is `noindex` , sends no referrer, isn't cached, uses a strict Content-Security-Policy, and HTML-escapes all text.
- **No AI rewriting:** the expert-sourced win text is shown unchanged.
- **One moment pipeline:** caregiver notes go through exactly the same pipeline as the parent's own (`createMoment()`): the safety check, then AI tagging, then the pattern check. A red flag is never sent to the AI and is logged as a safety event.
- **Limits:** 20 notes per link per day.

Built with: `caregivers` and `playbook_wins` tables plus `caregiver_id` and `logged_by` on `moments` , migration `0003_family_playbook.sql` , `services/playbook.ts` , `routes/playbook.ts` (parent API), `routes/publicPlaybook.ts` (caregiver page), `PUBLIC_BASE_URL` in `.env` , and the mobile `family.tsx` screen.

3. Smaller improvements

Change	Where	What it does
Ask other parents	Under every chat answer (Ask tab)	Opens a new Circles question with the parent's question already filled in and the right circle picked. After a 🗨️ it reads "Not quite right? Ask other parents" and is highlighted.
Share how it went	On every win card	Opens a "What worked" post with that win preselected, which feeds the "Parents like you" stats.
Parents like you	On every win card	"5 of 7 who tried this said it helped", once 3 or more parents have reported.
Inbox reply notices	Notifications tab	Circles replies appear next to daily ideas, and tapping one opens the post.
Coloured Story pebbles	Story tab, "Log at least 3 moments" card	The three progress pebbles use the welcome screen's colours with an ink outline. Pebbles for moments not logged yet are softer, so progress still reads.
New icons	App-wide	People, more (...) and heart icons in the app's hand-drawn style.
Fix: "5 and up" filter	Circle feed	The 5y+ age band is now URL-encoded; a raw + was read as a space and rejected.
Refactor: one moment pipeline	API	Saving a moment (safety, tagging, patterns) moved from the Story route into <code>createMoment()</code> , shared by the parent's journal and caregiver notes.

4. Quality work behind these features

- **26 new automated tests** (77 in total, all passing):
 - `test/community.test.ts` (18): anonymity, nickname scrubbing, PII rules, crisis handling, developmental worries, medication review, the harassment block, fail-closed moderation, reports and approval, blocking, reactions, rate limits, age bands and the nearby fallback, the guide reply's citations, no guide reply for uncovered questions, win stats, and the 5y+ band.
 - `test/playbook.test.ts` (8): default and entitlement rules for shared wins, unguessable links and the 5-person limit, page content and escaping, privacy (no email or other moments on the page), caregiver notes becoming tagged moments and counting towards patterns, red-flag handling, revoked links, other parents' caregivers, and rate limits.
- **Live-model checks with Groq:**
 - 10 hand-written Circles replies (support, venting, own child's diagnosis, "see a doctor", a dose, a diagnosis, an insult, spam, off-topic, shaming) were all handled as intended.
 - A full Family Playbook run went from link, to page, to note, to the tagged moment in Story.
- **AI mistakes log:** three new entries in [AI_MISTAKES.md](#):
 - The guide reply citing the wrong goal, fixed by a stricter match rule.
 - A moderation rule that would have blocked parents venting about themselves.
 - Three regex slips, found by tests.
- **Existing evals:** `npm run eval` still produces the same results as before these features.

5. Bug-fix pass

A review of the API and the mobile app turned up a set of real bugs, all fixed. 7 new tests cover the API side (84 tests in total, all passing).

Behaviour changes

Change	What it means
Locked wins stay locked in chat	Retrieval now only returns win 1 and goal overviews to free users, so Ask can't quote a paid win's script. Circles guide replies are public, so they only ever use free content. Subscribers get everything, as before.
Clearing the playbook shares nothing	Before, unselecting every win made caregiver links fall back to win 1 of each goal. Now that fallback applies only until the parent first chooses (<code>users.playbook_chosen_at</code> , migration <code>0004_playbook_chosen.sql</code>).
Production refuses dev secrets	With <code>NODE_ENV=production</code> , the API won't start while <code>JWT_SECRET</code> or <code>ADMIN_KEY</code> is still the public dev default. The admin key is compared in constant time.

API fixes

Bug	Fix
An empty <code>PATCH /me</code> or <code>PATCH /children/:id</code> returned 500	Treated as a no-op that returns the current row (404 if the child isn't yours).
<code>?today=2026-02-30</code> returned 500 from Postgres	Dates must be real calendar dates; impossible ones get a 400.
Marking an older Circles notice read returned an empty 500	<code>markNoticeRead</code> returns the notice itself instead of searching the newest 30.
"Load more" in a circle could skip posts	The feed cursor is now <code><timestamp>\ <id></code> , compared at millisecond precision with the id as tiebreak, because Postgres stores microseconds and JS dates don't.
Two registrations racing for one email returned 500	The unique-index error becomes a 409.
A chat closed mid-answer kept generating and wasn't costed	Disconnects are detected on the response socket, and an unfinished stream is logged to <code>llm_calls</code> with estimated tokens.
Approving a reply held for review sent no notice	The post's author now gets the same inbox notice a live reply would have sent.
The birth-year check was fixed at server start	"This year" is read on every request; a birth month later this year is rejected.

Mobile fixes

Bug	Fix
Wins stayed locked after subscribing (and unlocked after cancelling)	Checkout and cancel refresh every cached query, not just <code>/me</code> .
"Add moment" opened from a notification at launch could never be saved	The selected child is derived from the profile once it loads, instead of frozen as empty.
Story said "4 down, -1 to go"; Home promised a pattern that didn't exist	Both show "keep logging" once there are 3+ moments with no pattern; Home checks for a real pattern.
Silent failures	Error notes, with retry where it helps, for: circles failing to load (feed and composer), deleting a post or reply, removing a caregiver, deleting a moment (now confirmed first), and the profile failing to load. Failed queries also retry when the app returns to the foreground.
"Turn on" in Notifications left phone alerts off	It now turns phone alerts back on too, as Settings does, and reschedules straight away.
"I'll try this" promised a check-in with notifications off	It offers to turn notifications back on first.
"Ask about X" stacked a second tab bar	Uses <code>router.navigate</code> instead of <code>push</code> .
Switching accounts kept the previous profile's alerts	Signing in as a different user clears them; a guest creating an account keeps theirs.
A gateway or ngrok HTML error showed a JSON parse error	Shown as a readable "Request failed (status)"; chat network failures show "Can't reach the server".
Retrying "Create my plan" created a second guest account	Onboarding resumes from the step that failed.
iOS birth-date picker showed a date that didn't count as chosen	The date shown is now the date selected.
Sign-in opened in login mode before the profile loaded	It switches to "create account" once it knows you're a guest.

Not done yet (known, lower priority): rate limits on guest creation, chat and moments; count-then-insert races on child, caregiver and note limits; login timing; reactions and reports left behind when a post is deleted; keyboard offsets in chat; a fallback for browsers without a month picker.

6. Progress tracker: is it working?

Why. Wins tell parents what to try, but nothing showed whether it was working. "I'll try this" only set a reminder on the phone, and the answer came back as free text. After a hard week, memory says "nothing

works". The tracker records every attempt and its outcome so parents can see which wins help, when to switch, and that their effort is paying off.

What the parent sees

Where	What it shows
Win card , after I'll try this	"You're trying this. Once you have, how did it go?" with three chips: Helped , A bit , Not yet , plus "Add a note about what happened". A tap records the outcome and cancels the pending check-in.
Win card , once reported	"You tried this 4× · helped 3", above the "Parents like you" line.
Win card , next step	A coloured note chosen by the rules below. "Ask" includes an Ask ParentPal button.
Goal screen , top	Your progress : a headline, one dot per recent outcome (green helped, apricot a bit, grey not yet), counts, and once there's enough data two bars: Week 1 against This week .
Home , goal cards	"Helped 3 of the last 5 tries", or "Helped 3 of 3 this week, up from 1 of 4 in week 1" when things have improved.
Check-in notification	Opens the moment log with "Did it help?" chips; saving records the outcome and the note together.
Caregiver page (/p/:token)	An optional "Which idea?" picker. "It worked" or "It was tough" about a picked win counts as Helped or Not yet. With one shared win it's picked automatically.

Rules

- **Tries**: "I'll try this" creates a try. Tapping it again while a try is open reuses it. Only reported tries count, and an open try older than 7 days counts as "no answer". Only unlocked wins can be tracked.
- **Next step** (`adviceFor()` , no LLM):

Recent outcomes for the win	Note
Last 2 helped	"This is working. Keep going, and use the same words each time."
Last 2 helped a bit, none failed	"It's starting to help. Small changes count, so give it a week or two."
Last 3 didn't help, another win unlocked	"Not helping yet. Try win 2, "...?" (the next win, wrapping around, that isn't stuck too)
Last 3 didn't help, nothing else unlocked	"Not helping yet. Ask ParentPal what else might work..." (never a paywall pitch)

- **Trend** (`trendOf()`): the first 7 days of tries against the last 7 days. The two windows never overlap, and the trend appears only when each has at least 2 tries. Home only mentions it when it's an improvement.
- **Chat**: the family context sent to the LLM includes "Wins they tried and how it went", so answers can build on what's working.
- **Notes**: a note sent with an outcome goes through `createMoment()` , so the safety check, tagging and patterns work as before, and the moment is linked to the try.

Built with: `win_tries` table (migration `0005_win_tries.sql`), `services/progress.ts`, `routes/progress.ts`, `mine` (with `advice`) on each win in `GET /goals/:slug`, and the mobile `components/Progress.tsx`. The seed script now updates wins in place and deletes only removed ones, because deleting a win cascades to parents' tries and playbook choices.

Tests: `test/progress.test.ts` (11) covers reusing open tries, locked wins, ownership, counts per goal and win, open tries expiring, notes becoming linked moments (including the safety path), caregiver reports, account deletion, the chat context, every advice rule (including the locked-win case and wrap-around), and the trend windows. 95 tests in total, all passing.

7. Two children: a switcher for the whole app

Why. Onboarding let parents add a second child, but after that the app only looked at the first one. Home, Story and Ask showed the first child's name. Story listed both children's moments together under that name. **I'll try this** always counted the try against the first child, so a win tried with a younger sibling showed up as the older one's progress. Caregiver links, Circles labels and daily tips also used only the first child.

What the parent sees

Where	What it shows
Home, Story, Ask, All moments	Chips with each child's name, under the title (Ask: above the message box). Only shown with two children. The choice is shared by every screen and remembered on the phone.
Home and Story	The chosen child's name, age, moments, patterns and progress only.
New moment	Starts on the chosen child. Logging a moment for the other child switches the app to them.
Goal screen	"Trying these with [Mo] [Ada]". I'll try this , "You tried this 4x", the next-step note and the week 1 / this week card are all per child.
Check-in notification	One per win and child, naming the child ("Tap to note what happened with Ada"). Trying the same win with both children asks about each.
Ask	The answer is about the chosen child: their age, their recent moments and how their wins went. Suggested questions use their name.
Family playbook	"This link is for [Mo] [Ada]" when adding someone. Their page shows that child's name and their notes go into that child's Story. Each person in the list says who they're for.
Circles	The default age filter and the "Mom of a 2-year-old" label follow the chosen child. A new post asks "This post is about"; other parents still see only the age. Replies use the chosen child.
Daily tips	Children take turns by day, so each gets tips in their name and for their age.

Rules

- **Where the choice lives:** on the phone only (`parentpa1.activeChild`), not the server, so two parents on two phones can each look at a different child. It's cleared on sign out. If the saved child was removed, the app falls back to the first child.

- **API:** the affected routes take an optional `childId` (see [API](#)). Without it, lists cover every child and everything else uses the first child, exactly as before, so single-child families and older app builds are unchanged. Another family's child id returns 404.
- **Chat:** with a child chosen, the family context puts that child first, uses only their moments and progress, and adds "This question is about: Ada". An unknown id is ignored rather than failing a reply that has already started streaming.
- **Privacy:** Circles still replaces every child's nickname in posts, not just the chosen one.

Built with: `lib/children.ts` (`pickChild()`), a `childId` filter in `services/progress.ts`, `routes/story.ts`, `services/context.ts`, `services/community.ts` and `services/playbook.ts`, child rotation in `services/dailyTips.ts`; on mobile `useActiveChild()` in `lib/session.tsx`, `components/ChildSwitcher.tsx`, and query keys that include the child id. No migration: moments, patterns, tries and caregiver links already stored their child.

Tests: `test/siblings.test.ts` (7) covers separate progress and goal-screen counts per child, separate open tries, moments and patterns per child, rejecting another family's child on every route, caregiver links for the chosen child, Circles age bands, and the chat context focus. 111 tests in total, all passing.

Feature brief: Circles

In one line: anonymous groups of parents working on the same goal with children the same age, where every question gets an expert-grounded answer straight away and every "what worked" report makes the core plan more credible.

The problem

ParentPal gives expert-backed plans (goals → wins) and an AI that answers questions. Two needs are still unmet:

1. **"Is it just me?"** Much of what parents of young children feel is isolation and self-doubt. An AI can give the right advice but can't tell you that other parents are going through the same thing.
2. **"Will this actually work for my kid?"** A win from a guide is credible. A win that 5 of 7 parents with 2-year-olds say helped is persuasive.

Today parents take both needs to WhatsApp groups and Instagram comments. Those are noisy, not anonymous, not organised by age or problem, and full of unchecked medical advice. ParentPal can offer the safer version.

Why not just add a forum?

Parent forums fail in four predictable ways. Each became a design constraint:

Risk	What would happen	How Circles handle it
Cold start	Few users split across many rooms means empty feeds, so people leave	One circle per goal (not goal × age). Quiet age bands borrow from nearby ages . Every question gets an instant guide answer , so nothing goes unanswered.

Risk	What would happen	How Circles handle it
Bad advice	"Give 3mg melatonin", "sounds like ADHD"	AI and rule moderation before anything is published. Medicine doses and diagnoses go to review; insults and spam are rejected.
Crises posted in public	A parent in distress gets silence or judgement	The same safety rules as chat. Crisis posts are never published ; the parent sees helplines immediately.
Privacy	Children's names, photos and phone numbers in public	Text only. Numbers, emails and links are rejected. Children's names become "my child". A different nickname in every circle.

The guiding principle: **the community adds warmth and real-world evidence; the AI and the expert content keep it safe and useful.**

How a parent uses it

1. **Open the Circles tab.** Your circles (from the goals you picked) come first, e.g. *Handling tantrums*, with how active each one is this week.
2. **Browse your child's age.** The feed opens on your child's age band ("2 to 3"). You can switch to any age or "All ages".
3. **Ask a question.** For example, *"My child screams every time we leave the park. How do you handle it?"* You post as "Gentle Robin · Mom of a 2-year-old", never your name. Within seconds a **ParentPal guide** reply appears, built from the expert content and citing the win it came from. Other parents reply below it.
4. **React.** "Same here" (solidarity) and "Helpful" (useful). No downvotes, no follower counts, no profiles.
5. **Share what worked.** After trying a win, tap **Share how it went** on the win card. Pick "It helped", "Helped a bit" or "Didn't help" and add a tip. Once 3 parents have reported, the win shows **"Parents like you: 5 of 7 said it helped"**.
6. **Get pulled back in.** Replies to your post show up in the Notifications inbox, at most one notice per post per day.
7. **Stay in control.** Delete your own posts. Report or block anyone. Blocking hides a parent everywhere, and they're never told.

Other entry points: **"Ask other parents"** under any chat answer (it prefills your question, especially useful after a 🗨️), and **"Share how it went"** on every win.

What makes it different

- **AI is the first responder, not the only one.** Every question is answered within seconds from vetted content, with sources. Parents add experience on top. If the content doesn't cover the question, the guide stays silent instead of guessing.
- **The community feeds the product.** "What worked" reports turn into social proof on the wins, which supports the core loop: try a win, see results, stick with the plan, convert to paid.
- **Safety is built in from the start.** Moderation runs before publishing, fails closed (if the AI is down, posts wait for review) and is covered by automated tests.

- **Anonymous by design.** Parents share the embarrassing stuff (yelling, "I felt like a bad mom") because nobody knows who they are.

Safety and moderation at a glance

What a parent writes	What happens
Support, venting, self-criticism, a parent's own child's diagnosis, "see your pediatrician"	Published immediately
A medicine or dose, a diagnosis of someone else's child, off-topic content	Held for review; only the author sees it
Insults, shaming, selling or "DM me"	Rejected, with a kind explanation
Phone number, email, link, @handle	Rejected: "Circles are anonymous, please remove..."
Emergency, self-harm, abuse	Not published; helplines shown immediately
A developmental worry ("not talking yet at 2")	Published, with a "talk to your pediatrician" card pinned and no AI answer
Reported by 3 different parents	Hidden until a moderator decides

Checked against the live model (Groq `gpt-oss-120b`) with 10 hand-written replies covering each row above: all 10 were handled as intended.

Metrics to watch

Proposed, not yet instrumented:

- **Activation:** % of weekly active parents who open Circles, and % who post or reply.
- **Answer speed:** % of questions with a guide reply (target: most on-topic questions), and time to first parent reply.
- **Product link:** number of "what worked" reports per win, and whether parents who see "Parents like you" stats try more wins or convert to paid more often.
- **Health:** % of posts held for review, reports per 100 posts, and how long items wait in the review queue.
- **Retention:** D7/D30 retention of parents who posted vs. those who didn't.

Rollout suggestion

1. **Seed honestly.** Before launch, the team posts real questions it has heard from parents, labelled as from the ParentPal team, never fake parents.
2. **Start with the 2–3 most-picked goals,** and widen once feeds are active.
3. **Daily moderation owner.** Someone checks the review queue every day. It's an API endpoint today; a simple admin screen is the first follow-up.
4. **Watch the review rate.** If too many good posts get held, tune the moderation prompt with real examples.

Effort and status

Built end to end: API, database migration, moderation, mobile screens, and 18 automated tests covering anonymity, moderation, crisis handling, reports, blocks, rate limits, the guide reply and win stats. Engineering details are in [How Circles work](#).

Next steps, in priority order: 1. Admin screen for the review queue. 2. Push notifications for replies. 3. A weekly AI digest per circle ("This week, parents of 2-year-olds found..."). 4. Semantic search (pgvector) so the guide answers more questions.

Feature brief: Family Playbook

In one line: every adult who looks after the child gets the same plan and the same words, on a link that needs no app, and their notes flow back into the parent's Story.

The problem

ParentPal coaches **one parent**. But a toddler's day, especially in Indian joint families, is shared between several adults: Nani or Dadi, the other parent, a nanny, a daycare teacher. Behaviour plans fail when the adults respond differently. Mum holds the screen-time limit, Nani hands over the phone, and the child learns that pushing harder works.

Every goal guide in ParentPal repeats the same point: **consistency** is what makes a win stick. Until now the app gave the parent no way to get that consistency from the rest of the family.

How it works for the parent

1. **Pick what to share.** On a win, tap **Send to family**, or open *Family playbook* from Home or Profile. Until you choose, it shares win 1 of each of your goals. You can share up to 4 wins, or none. Fewer is easier to stick to.
2. **Add a person.** Type what your child calls them ("Nani", "Papa", "Aunty Meena") and pick a relationship. Tap **Create link and share**. The phone's share sheet opens with a warm, ready-to-send WhatsApp message and the private link.
3. **See that it's working.** Each person shows "Opened 2h ago · 3 notes". Their notes appear in Story as **"Logged by Nani"** and count towards the 3 moments needed for a pattern.
4. **Stay in control.** Remove a person and their link stops working at once. Notes they already sent stay in the journal.

How it works for the grandparent

They tap the link on WhatsApp, and a simple page opens. There's nothing to install and no sign-up.

- "Hi Nani! Here's what we're trying with Aarav right now, shared by Priya (Mom)."
- One card per win: **What to do**, **Say this** (the exact words, in large type), and **What to expect**.
- **How did it go?** Two big buttons, "It worked" or "It was tough", plus an optional line of text, then **Send note**.

Why it matters for the business

- **Better outcomes:** the plan only works if everyone follows it. This is the missing piece between the advice and real behaviour change at home.
- **More data, better AI:** patterns now draw on notes from the whole household, not just one parent's memory at the end of the day. A grandparent with the child all afternoon sees things the parent doesn't.
- **Distribution:** every parent sends 1–3 links on WhatsApp, the channel Indian families already use. Every grandparent, nanny and teacher who opens one sees ParentPal's guidance working, at no acquisition cost. Teachers and nannies look after many children, which makes them a natural channel to more parents.
- **Retention:** notes from family arriving in Story give the parent a reason to come back.

Trust, safety and privacy

Concern	What the playbook does
Someone guesses a link	Links carry 32 random bytes and can be turned off at any time (it returns 410 Gone).
The caregiver sees too much	The page shows only the child's nickname, the parent's first name and role, and the shared wins. Never chat, other moments, patterns or email.
The advice gets changed	The page shows the win's own expert-sourced text unchanged . No AI rewrites it, so nothing can drift from the source.
A caregiver writes something alarming	The same safety rules as the app. A red-flag note (e.g. "he swallowed a battery") is never sent to the AI; the caregiver sees helplines straight away, and the parent sees the note in Story.
The page gets indexed or leaks	<code>noindex</code> , <code>no-referrer</code> , a strict Content-Security-Policy, <code>no-store</code> caching. Free-text fields are HTML-escaped.
Spam	20 notes per link per day. At most 5 people per family.

Metrics to watch

Proposed, not yet instrumented:

- % of active parents who share at least one link, and links per parent.
- % of links opened within 24 hours, and % of opened links that send at least one note.
- Notes per week from caregivers vs. parents, and time to first pattern for families with caregivers.
- Retention (D30) of parents with an active caregiver vs. without.
- Downloads attributed to playbook pages (needs a "Get ParentPal" link once the app is in the stores).

What's next

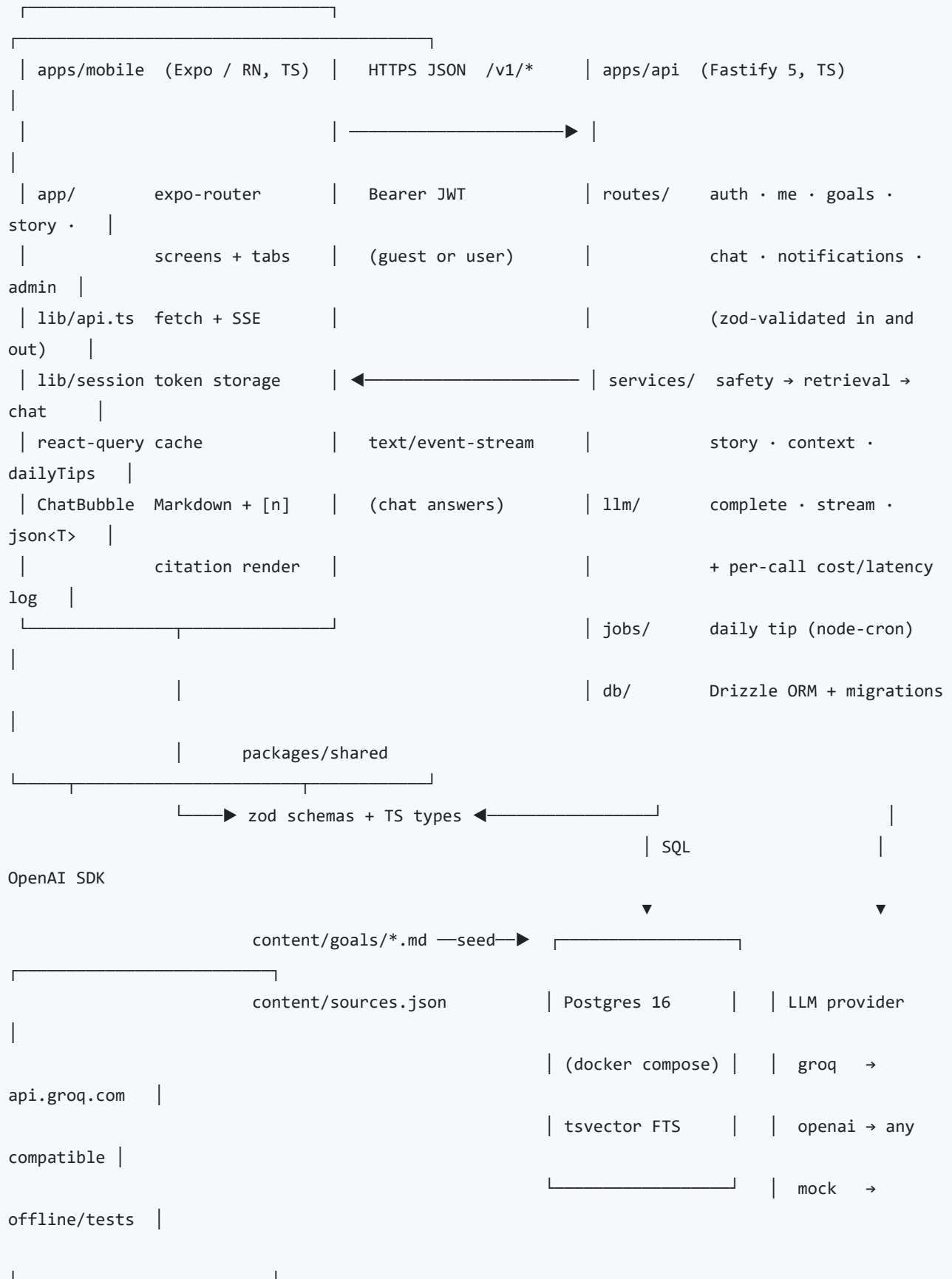
1. **Languages:** Hindi, Telugu, Tamil and others for grandparents. The page and data model are ready for it; it needs a reviewed translation step.
2. A short daily "today's focus" message to caregivers.
3. Showing the parent which wins caregivers find hard.

4. A "Get ParentPal" link on the page once there's a store listing.

Engineering details are in [How the Family Playbook works](#).

Architecture

An npm-workspaces monorepo: one Expo app, one Fastify API, one shared schema package, Postgres, and an LLM behind a small provider interface.



Path	Contents
apps/api	Fastify API: routes, services (chat, retrieval, safety, story, daily tips), LLM adapters, Drizzle schema and migrations, seed script, tests, eval runner
apps/mobile	Expo / React Native app (iOS, Android, web) using expo-router
packages/shared	zod schemas and TypeScript types shared by the API and the app, so request/response shapes can't drift
content/	Goals and wins as Markdown, plus <code>sources.json</code> . See content/README.md .
evals/	Eval cases (<code>cases.json</code>) and run results
scripts/tunnel-api.mjs	Exposes the local API on an ngrok static domain for testing on a real phone

API layers

Requests flow **route** → **service** → **db / llm**:

- **Routes** (`apps/api/src/routes`) handle HTTP only: auth (`requireUser`), zod validation and response shaping. Everything is mounted under `/v1`.
- **Services** (`apps/api/src/services`) hold the product logic: `safety` (red-flag rules), `retrieval` (Postgres FTS over content chunks), `context` (child age, goals and recent moments the LLM may see), `chat` (clarify / answer / citation validation), `story` (moment tagging, pattern insights) and `dailyTips`.
- **LLM** (`apps/api/src/llm`) is the only code that talks to a model. Every call states a `purpose` and a deterministic `mock()` output, and is logged to `llm_calls`.
- **DB** (`apps/api/src/db`) is the Drizzle schema plus SQL migrations. Every user-owned table cascades from `users`.

Data model

Group	Tables
Accounts	<code>users</code> , <code>children</code> , <code>subscriptions</code>
Content	<code>goals</code> , <code>wins</code> , <code>sources</code> , <code>content_chunks</code> , <code>user_goals</code>
Story	<code>moments</code> , <code>patterns</code> , <code>pattern_moments</code>
Progress	<code>win_tries</code> (one row per attempt at a win; <code>outcome</code> is null until reported)
Chat	<code>conversations</code> , <code>messages</code> , <code>message_feedback</code> , <code>bookmarks</code>
Ops	<code>notifications</code> , <code>llm_calls</code> , <code>safety_events</code>

Mobile app

- **Routing**: `expo-router` with file-based routes in `apps/mobile/src/app`. Tabs are Home, Story, Ask, Notifications and Profile; onboarding, goal detail, paywall, bookmarks and settings are stack screens.

- **Notifications:** `lib/notificationPlan.ts` is a pure planner that picks at most one notification per day for the next 3 days (priority: check-in, log reminder, daily idea). `lib/notifications.ts` holds the on-device prefs and check-ins, asks for permission in context, schedules with `expo-notifications` on every app open, and routes taps to the right screen. Nothing goes through a push server, so notification text never leaves the phone; if the app isn't opened for ~3 days, notifications pause. `lib/expoNotifications.ts` imports only the parts of `expo-notifications` the app uses, because importing the package root crashes Android Expo Go.
- **Data:** `lib/api.ts` wraps `fetch` for JSON calls and `expo/fetch` for the chat SSE stream. Server state is cached with `react-query`, and the JWT lives in secure storage (`lib/session.tsx`).
- **Chat rendering:** `ChatBubble` renders the small slice of Markdown models produce (bold, bullet and numbered lists, headings) with the app's own fonts, and turns `[n]` markers into citation tags that match the "Based on" source list. Copy/share strips the Markdown.

How the chat answers a message

1. **Safety gate:** regex/phrase rules per category. A match returns a fixed safety message and logs a `safety_events` row that stores the category only, never the text. The LLM is not called.
2. **Retrieve:** Postgres full-text search over content chunks, top 3, boosted for the user's active goals. Chunks below a minimum score are never shown to the model or cited. Free users only get win 1 and goal overviews, so the chat can't quote a locked win.
3. **Clarify:** if the question is short or generic *and* retrieval confidence is low, the API asks one clarifying question with 2–3 options instead of answering.
4. **Answer:** a system prompt plus child context plus retrieved chunks, streamed back over SSE. The server checks that every citation id the model gives was actually retrieved and drops any that weren't.
5. **Log:** provider, model, purpose, tokens, latency and cost go to `llm_calls` . Prompts and outputs are not stored.

How notifications work

Why they exist. The app's core loop is *pick a goal* → *try a win* → *log what happened* → *get pattern insights*. Parents drop off most between trying something and logging it, so notifications serve that loop rather than just pulling people back into the app. Each one either delivers an idea at a moment it can be used or asks for the one-line note that makes patterns possible.

The three kinds

Kind	When it fires	What it says	Tapping it opens
Check-in	8:30 am the day after the parent taps I'll try this on a win	<i>How did "Name the feeling" go?</i>	The moment log, titled "How did it go?" with <i>Tried "Name the feeling"</i> . filled in and "Did it help?" chips that record the outcome
Logging reminder	7:30 pm on the 3rd day without a logged moment, once per quiet streak	<i>Anything happen with Mo lately?</i>	An empty moment log

Kind	When it fires	What it says	Tapping it opens
Daily idea	At the chosen time. Smart (default): sleep ideas 6:30 pm, picky-eating ideas 4:30 pm, everything else 8:00 am. Or a fixed Morning / Midday / Evening.	The day's tip from the server, e.g. <i>Today's idea: The 2-minute warning</i>	That goal's page (and marks the tip read)

Rules that keep them welcome

- At most **one notification per day**. If several are due, the check-in wins, then the logging reminder, then the daily idea.
- Nothing is scheduled between 9 pm and 7 am, and times that have already passed today are skipped.
- The OS permission prompt never appears on launch. It only appears after the parent taps **Turn on** on an explainer card in the Notifications tab, or agrees to a check-in from **I'll try this**. If the OS has already refused, the app offers to open system settings instead.
- **Settings** → **Notifications** has one account-level **Notifications** switch, stored on the server (`users.notifications_enabled`) so it works on every platform, including web and Android Expo Go. Off: no new daily ideas are created (the morning job, the inbox and `upcoming` all skip the user), the tab badge disappears, phone alerts are cancelled, and the Notifications tab shows a "Notifications are off" card with a Turn on button. Under it, where phone alerts are possible, **Phone alerts** and each kind can be switched separately.
- Signing out cancels everything scheduled for that profile.

How a notification gets to the phone

Notifications are scheduled **on the device**, not sent from the server:

```

app opens / returns to foreground / setting changes / moment logged / "I'll try this"
    |
    ▼
syncNotifications()                                apps/mobile/src/lib/notifications.ts
  1. load prefs + pending check-ins (device storage)
  2. fetch  GET /v1/me                                → child's nickname, sign-up date
           GET /v1/notifications/upcoming → tips for today + 2 days (created on demand)
           GET /v1/moments                  → date of the last logged moment
  3. planNotifications(...)                        apps/mobile/src/lib/notificationPlan.ts
       for each of the next 3 days: pick ≤ 1 by priority, drop past times
  4. cancel all scheduled → schedule the plan with expo-notifications (date triggers)
    |
    ▼
OS shows it at the planned time → tap → useNotificationTaps() routes to the right screen

```

If the API can't be reached during a sync, the previous schedule is kept. Syncs are queued, so overlapping triggers never interleave cancel and schedule.

Why on-device rather than server push. It needs no push credentials, Firebase project or server-side token storage, and it works in iOS Expo Go. No notification text passes through Apple, Google or Expo

push servers. The trade-off is that the schedule only reaches 3 days ahead, so if the app isn't opened for about 3 days, notifications pause. The kinds and planner would carry over unchanged to server push later.

Server side. Tips come from the same generator as the daily job (`apps/api/src/services/dailyTips.ts`): one per user per day (enforced by a unique index), rotating through the wins of the user's goals. `GET /notifications/upcoming` creates any missing tips for the requested days on demand, so the phone can schedule them in advance. `from` is the phone's local date and must be within a day of the server's, and `days` is capped at 3 to bound LLM cost. The inbox (`GET /notifications?today=...`) hides tips dated after the phone's today, so tomorrow's idea doesn't show up early.

Where it lives

File	Role
<code>apps/mobile/src/lib/notificationPlan.ts</code>	Pure planner: types, times, priority, one-per-day rule. No React Native imports, so it can run under Node.
<code>apps/mobile/src/lib/notifications.ts</code>	Prefs and check-ins store, permission flow, scheduling, foreground sync and tap routing hooks
<code>apps/mobile/src/app/(tabs)/notifications.tsx</code>	Inbox plus the "Turn on" explainer card
<code>apps/mobile/src/app/goal/[slug].tsx</code>	I'll try this button on each unlocked win
<code>apps/mobile/src/app/moment/new.tsx</code>	"How did it go?" mode when opened from a check-in
<code>apps/mobile/src/app/settings.tsx</code>	Notification settings
<code>apps/api/src/routes/notifications.ts</code> , <code>services/dailyTips.ts</code>	Inbox, upcoming tips, tip generation

Where it runs. iOS (Expo Go or a build) and Android **development or release builds**. Android Expo Go (SDK 53+) ships `expo-notifications` without the channel support Android needs to show any notification, so there the app detects Expo Go, hides the notification controls and explains why in Settings. The rest of the app works normally. Notifications are also hidden on web, where on-device scheduling isn't available.

Trying it out. Run the app on an iPhone with Expo Go, or on Android with a development build (`npm expo run:android` , or an EAS development build). Turn notifications on from the Notifications tab, then open a goal and tap **I'll try this**. The check-in arrives the next morning at 8:30. Today's idea is already in the inbox.

How Circles work

Why a constrained version. Parents want to hear "it's not just you" from other parents, which the AI can't really provide. But open parent forums fail in predictable ways: empty rooms early on, medical misinformation, crises posted in public, and children's names and photos. Circles are designed around those four problems.

Problem	What Circles do
Empty rooms	One circle per goal (not goal × age). Posts carry an age band, and a quiet band shows posts from nearby ages . Every question gets an instant guide reply built with the chat pipeline: same prompt, a stricter score threshold, only chunks from the circle's own goal, and dropped entirely if it can't cite.
Misinformation	<code>services/moderation.ts</code> runs on every post and reply. Dose and medicine mentions (rules) and diagnoses of someone else's child (LLM) go to review : only the author sees them until a moderator approves. Insults, shaming and spam are rejected with a reason. If the LLM is down, items wait for review (fail-closed).
Crises	The same red-flag rules as chat. Emergencies, self-harm and abuse are never published ; the author sees the safety card instead. A developmental worry is published with the "talk to your pediatrician" card pinned and no guide reply.
Identity	Phone numbers, emails, links and @handles are rejected. The author's own children's nicknames are replaced with "my child". Pseudonyms come from <code>hash(secret, circle, user)</code> , so the same parent can't be linked across circles. Only signed-up accounts can post, react or report, which limits spam. Guests can read.

Three reports from different parents hide an item until a moderator decides; once a moderator approves it, reports alone can't hide it again. Blocking hides a parent's content everywhere, for the blocker only. Limits: 5 posts and 30 replies a day. Reply notices (at most one per post per day) appear in the Notifications inbox.

File	Role
<code>apps/api/src/services/community.ts</code>	Feed, posts, replies, reactions, reports, blocks, guide reply, win stats, inbox notices
<code>apps/api/src/services/moderation.ts</code>	PII rules, nickname scrub, safety gate, medication rule, LLM verdict
<code>apps/mobile/src/app/(tabs)/circles.tsx</code> , <code>circle/[goal].tsx</code> , <code>post/[id].tsx</code> , <code>post/new.tsx</code>	Circles tab, feed, post and composer

How the Family Playbook works

- **Data:**
- `caregivers` holds the name, relation, a random token, `revokedAt` and `lastOpenedAt`.
- `playbook_wins` holds the parent's chosen wins. `users.playbook_chosen_at` records that they have chosen, so an empty choice shares nothing instead of the win-1 defaults.
- `moments.caregiverId` and `moments.loggedBy` record who logged a moment. The name is a snapshot, so it survives removing the caregiver.
- **Entitlement:** only wins the parent can see can be shared (win 1, or any win with a subscription). If a subscription lapses, locked wins drop out of the page.
- **One moment pipeline:** `createMoment()` in `services/story.ts` is used by both the parent's Story and the caregiver page. It runs the safety check, then LLM tagging, then the pattern check, so caregiver notes behave exactly like the parent's.

- **Public page:** `routes/publicPlaybook.ts` serves server-rendered HTML at `/p/:token` and accepts notes at `POST /p/:token/log` (optional `winId` to count a quick note as a try). It lives outside `/v1` and needs no auth; the link is the key.
- **Link host:** set by `PUBLIC_BASE_URL`. If that's empty it uses `https://<NGROK_DOMAIN>`, then `http://localhost:<PORT>`, so links work from other phones during development. ngrok's free plan shows its own warning page the first time a browser opens a link. A real deployment wouldn't have this.

File	Role
<code>apps/api/src/services/playbook.ts</code>	Shared wins, caregivers, links, the public view, caregiver notes, rate limit
<code>apps/api/src/routes/playbook.ts</code> , <code>routes/publicPlaybook.ts</code>	Parent API; caregiver page and note endpoint
<code>apps/mobile/src/app/family.tsx</code>	Family playbook screen (choose wins, add, share, remove)

LLM providers

`apps/api/src/llm/` exposes `complete()`, `stream()` and `json<T>(schema)` with three adapters behind one `LLMProvider` interface:

- **groq** (default for real answers): used when `GROQ_API_KEY` is set. Reuses the OpenAI adapter pointed at Groq's OpenAI-compatible API (`https://api.groq.com/openai/v1`), default model `openai/gpt-oss-120b` (`GROQ_MODEL`). `gpt-oss` always reasons, so keep `LLM_REASONING_EFFORT=low` and some `LLM_THINKING_TOKEN_HEADROOM` so reasoning doesn't use up the small per-call caps.
- **openai** : used when `OPENAI_API_KEY` is set (and no Groq key). Works with any OpenAI-compatible endpoint via `OPENAI_BASE_URL`.
- **mock** : deterministic and template-based. It is used when no key is set and always in tests, so the whole app runs offline at no cost.

Set `LLM_PROVIDER` to force one. Each row in `llm_calls` records the provider, model, purpose, tokens, latency and cost, and `GET /v1/admin/costs` summarizes them.

Getting started

Runs on Windows, macOS and Linux. No API keys are needed: without one, the app uses a built-in mock LLM, so every screen works offline.

Quick start (if you already have Node 22.9+ and Docker running):

```
git clone https://github.com/kanugurajesh/parentpal.git
cd parentpal
npm install
cp .env.example .env
npm run dev
```

Then press `w` in the terminal to open the app in your browser. The steps below explain each part, and [Troubleshooting](#) covers the usual problems.

1. Install the prerequisites

Tool	Version	Check with	Notes
Node.js	22.9 or newer (tested on 24)	<code>node -v</code>	The scripts use <code>--env-file-if-exists</code> , which older Node versions don't have.
npm	comes with Node	<code>npm -v</code>	The repo is an npm workspace; use npm, not yarn or pnpm.
Docker Desktop	Compose v2.17+	<code>docker compose version</code>	Runs Postgres. Start Docker before running any script.
Git	any	<code>git --version</code>	

Optional:

- **To see the app on a phone:** the [Expo Go](#) app. The project uses **Expo SDK 57**, so install the current Expo Go from the app store.
- **For an emulator:** Android Studio (Android emulator) or Xcode (iOS simulator, macOS only).
- **For real AI answers:** a free [Groq API key](#), or an OpenAI key.

2. Clone and install

```
git clone https://github.com/kanugurajesh/parentpal.git
cd parentpal
npm install
```

`npm install` at the root installs the API, the mobile app and the shared package together. It takes a few minutes the first time.

3. Create your `.env`

```
cp .env.example .env          # Windows PowerShell: Copy-Item .env.example .env
```

The defaults work as they are. Things you may want to change:

Setting	When to change it
<code>JWT_SECRET</code>	Set any long random string. Required in production; fine as is locally.
<code>GROQ_API_KEY</code> (or <code>OPENAI_API_KEY</code>)	Add one for real AI answers instead of the mock.
<code>RESEND_API_KEY</code>	Only to send real password-reset emails. Without it, the code is printed in the API terminal.

All settings are listed under [Environment variables](#).

4. Run it

```
npm run dev
```

This one command:

1. starts Postgres in Docker on port **5433** (it creates the main and test databases the first time),
2. runs the database migrations,
3. seeds the parenting content (14 goals, 26 wins),
4. starts the API on **http://localhost:4000** and the Expo dev server on port **8081**, side by side.

It's ready when the API has logged that it's listening on port 4000 and Expo shows its QR code. Then open the app:

Where	How
Browser	Press <code>w</code> in the terminal, or run <code>npm run dev:web</code> instead of <code>npm run dev</code> .
Phone (same Wi-Fi)	Scan the QR code with Expo Go (Android) or the Camera app (iOS). See Running on a phone .
Android emulator	Start the emulator, then press <code>a</code> .
iOS simulator (macOS)	Press <code>i</code> .

Check that the API is up: open `http://localhost:4000/health`. It shows which LLM is in use (`mock` , `groq` or `openai`).

Stop everything with `Ctrl+C` . Postgres keeps running in the background; `npm run db:down` stops it.

5. Try it out

There are no demo logins: the app is guest-first, so every flow starts from a fresh profile.

1. **Onboarding:** add a child (and optionally a second one), pick 1–2 goals, then close the paywall to use the free tier.
2. **Story:** log three moments about the child. A pattern appears after the third.
3. **Ask:** ask a question, e.g. "How do I handle tantrums at bedtime?". Answers cite their sources.
4. **Goals:** open a goal, tap **I'll try this** on win 1, then record how it went.
5. **Account:** Profile → **Create account**. To test **Forgot password**, sign out, tap *Forgot password?*, and copy the 6-digit code from the API terminal.
6. **Two children:** Profile → **Edit** → add a child, then switch between them on Home.
7. **Unlock everything:** choose any plan on the paywall. It's a fake subscription; no payment is taken.

The Circles review queue and LLM costs are on admin routes that need the `x-admin-key` header (default `dev-admin-key`). See [API](#).

Running on a phone

A phone can't use `localhost` , because on the phone that means the phone itself.

- **Same Wi-Fi (the usual case):** nothing to configure. In development the app reaches the API through the same computer the Expo dev server runs on. If the app says it can't reach the server:
- allow Node.js through your firewall on **private networks** (Windows asks the first time), and
- if it still fails, create `apps/mobile/.env.local` containing `EXPO_PUBLIC_API_URL=http://<your computer's LAN IP>:4000`, then restart `npm run dev`. Expo only reads env files from `apps/mobile`, not the root `.env`.
- **Different networks, or Wi-Fi that blocks devices from seeing each other:** use the ngrok tunnel. Put `NGROK_AUTHTOKEN` and a free static `NGROK_DOMAIN` from [ngrok](#) in the root `.env`, set `EXPO_PUBLIC_API_URL=https://<NGROK_DOMAIN>` in `apps/mobile/.env.local`, then run `npm run dev:tunnel`.
- **Notifications** don't work in Expo Go on Android (Expo's limit, not this app's). Use iOS, or an Android development build (`npm run -w @parentpal/mobile android`), to test them.

Running the tests

```
npm test          # starts Postgres if needed, then runs the 111 API tests
npm run typecheck # type-checks every workspace
npm run eval      # AI eval suite (mock LLM); add -- --judge with a real key
```

Tests use their own database (`parentpal_test`) and always use the mock LLM, so they're free, offline and never touch your dev data.

Troubleshooting

Problem	Fix
<code>npm run dev</code> fails at <code>docker compose</code>	Docker isn't running. Start Docker Desktop, wait until it says it's running, and try again.
bad option: <code>--env-file-if-exists</code>	Node is older than 22.9. Upgrade Node and run <code>npm install</code> again.
port is already allocated (5433), or <code>EADDRINUSE</code> (4000)	Another Postgres or an earlier run is using the port. Stop it, or change the port in <code>docker-compose.yml</code> and <code>DATABASE_URL</code> / <code>TEST_DATABASE_URL</code> , or <code>PORT</code> .
Expo says port 8081 is in use	An earlier Expo is still running. Close it, or press <code>y</code> to use another port.
The health check shows <code>openai</code> or <code>groq</code> though you set no key	A key is set in your shell's environment, and shell variables win over <code>.env</code> . Unset it, or set <code>LLM_PROVIDER=mock</code> in <code>.env</code> .
Phone shows "Can't reach ParentPal's server"	See Running on a phone : firewall first, then <code>EXPO_PUBLIC_API_URL</code> .
Expo Go says the project needs a newer SDK	Update Expo Go from the app store (the project uses SDK 57).
<code>ERR_NGROK_334</code>	An earlier tunnel is still running. Stop it, then run <code>npm run dev:tunnel</code> again.
<code>relation ... does not exist</code>	Migrations didn't run. Run <code>npm run setup</code> .

Problem	Fix
Tests fail with <code>database "parentpal_test" does not exist</code>	The test database is only created with a fresh Docker volume. Run <code>docker compose exec db createdb -U parentpal parentpal_test</code> , or reset the database (below).
You want a clean database	<code>docker compose down -v</code> deletes all local data, then <code>npm run setup</code> recreates it.

Environment variables

Variable	Default	Purpose
<code>PORT</code>	<code>4000</code>	API port
<code>DATABASE_URL</code> / <code>TEST_DATABASE_URL</code>	local docker DBs on <code>5433</code>	Postgres connections (the test DB is created by <code>apps/api/db-init</code>)
<code>JWT_SECRET</code>	dev placeholder	Signs auth tokens. Change it. The API won't start in production with the default.
<code>ADMIN_KEY</code>	<code>dev-admin-key</code>	Required as the <code>x-admin-key</code> header on <code>/v1/admin/*</code> and <code>/v1/dev/*</code> . The API won't start in production with the default.
<code>LLM_PROVIDER</code>	<code>auto</code>	<code>groq</code> , <code>openai</code> or <code>mock</code> . Auto-selects <code>groq</code> , then <code>openai</code> , by which key is present.
<code>GROQ_API_KEY</code> , <code>GROQ_MODEL</code>	<code>openai/gpt-oss-120b</code>	Groq config
<code>OPENAI_API_KEY</code> , <code>OPENAI_MODEL</code> , <code>OPENAI_BASE_URL</code>	<code>gpt-4o-mini</code>	OpenAI or any OpenAI-compatible endpoint
<code>LLM_REASONING_EFFORT</code>	<code>empty</code>	Sent as <code>reasoning_effort</code> (<code>low</code> recommended for <code>gpt-oss</code>)
<code>LLM_THINKING_TOKEN_HEADROOM</code>	<code>0</code>	Extra output tokens per call for reasoning models (<code>512</code> in <code>.env.example</code>)
<code>LLM_PRICE_INPUT_PER_M</code> , <code>LLM_PRICE_OUTPUT_PER_M</code>	<code>0.15</code> , <code>0.60</code>	USD per 1M tokens for cost logging. Check against current pricing.
<code>DAILY_TIP_CRON</code>	<code>0 8 * * *</code>	Server job that fills the inbox with each user's tip for the day (server local time). Phones also fetch upcoming tips themselves.
<code>EXPO_PUBLIC_API_URL</code>	<code>http://localhost:4000</code>	API base URL for the app (set in <code>apps/mobile/.env.local</code>)
<code>NGROK_AUTHTOKEN</code> , <code>NGROK_DOMAIN</code>	<code>empty</code>	Used by <code>npm run dev:tunnel</code>

Variable	Default	Purpose
RESEND_API_KEY , EMAIL_FROM	empty, ParentPal <onboarding@resend.dev>	Sends password-reset codes through Resend . Empty: in development the code is printed in the API log; in production nothing is sent and a warning is logged.
RATE_LIMITS	on	off disables rate limits (local load testing only). Tests run without them except <code>rate-limit.test.ts</code> .
TRUST_PROXY	auto	Trust <code>X-Forwarded-For</code> so IP limits see the phone, not the proxy. Auto: on when <code>NGROK_DOMAIN</code> is set. Only turn it on behind a proxy you control.

Scripts

Run from the repo root:

Command	What it does
<code>npm run dev / npm run dev:web</code>	Full stack (native / web)
<code>npm run dev:tunnel</code>	Full stack with the API and Expo exposed through tunnels, for a phone on any network
<code>npm run tunnel:api</code>	Only the ngrok tunnel to the local API
<code>npm run setup</code>	Start the DB, migrate, seed
<code>npm run db:up / npm run db:down</code>	Start / stop Postgres
<code>npm test</code>	API test suite (vitest against a dockerized test DB, mock LLM)
<code>npm run eval</code>	Run the eval suite (add <code>-- --judge</code> for an LLM grounding judge; needs a real provider)
<code>npm run typecheck</code>	Type-check all workspaces
<code>npm run -w @parentpal/api db:seed</code>	Re-seed after editing <code>content/</code>
<code>npm run -w @parentpal/api jobs:daily-tips</code>	Run the daily tip job once

API

All routes live under `/v1`, take and return JSON validated with zod, and need a Bearer JWT unless noted. The main groups:

- **Auth:** `POST /auth/guest`, `/auth/register` (upgrades the guest), `/auth/login`, `/auth/forgot` (emails a 6-digit code; always answers `{ ok: true }`), `/auth/reset` (code + new password; signs in

and signs out other devices)

- **Profile:** GET|PATCH|DELETE /me (DELETE hard-deletes everything through cascades), PUT /me/goals , POST|PATCH|DELETE /children[:id]
- **Goals:** GET /goals , GET /goals/:slug (locked wins return only their title), GET /advisors , POST|DELETE /subscription (fake)
- **Story:** GET|POST /moments , DELETE /moments/:id , GET /patterns , POST /patterns/generate
- **Progress:** POST /tries (start or reuse an open try), POST /tries/:id/outcome (helped , somewhat or didnt , plus an optional note that becomes a moment), GET /progress (per-goal counts, recent outcomes, trend, per-win counts, open tries)
- **Chat:** GET /chat , DELETE /chat (starts a fresh conversation; bookmarked replies are kept), GET /chat/starters , GET /chat/topics , POST /chat/messages (SSE: meta → delta * → done), feedback and bookmark routes, GET /bookmarks
- **Notifications:** GET /notifications?today=YYYY-MM-DD (inbox; hides tips dated after the device's today), GET /notifications/upcoming?from=YYYY-MM-DD&days=1-3 (creates and returns the next days' tips for on-device scheduling; from must be within a day of the server's date), POST /notifications/:id/read
- **Family playbook:** GET /playbook , PUT /playbook/wins , POST /caregivers (returns the link and a ready-to-send message), DELETE /caregivers/:id (turns the link off). Public, no auth: GET /p/:token (HTML page), POST /p/:token/log .
- **Circles:** GET /circles , GET|POST /circles/:goal/posts (?band=2-3y|all&cursor= , where cursor is the previous page's opaque nextCursor), GET|DELETE /posts/:id , POST /posts/:id/replies , DELETE /replies/:id , POST /reactions (toggles), POST /reports , POST /blocks . Writes need a signed-up account.
- **Ops** (needs x-admin-key): GET /admin/costs?days=30 (LLM cost and latency breakdown), POST /dev/run-daily-tips , GET /admin/community/queue , POST /admin/community/:type/:id (approve or remove)

GET /health (no auth) reports which LLM provider is active.

Two children: GET /moments , /patterns , /progress , /goals/:slug , /circles and /chat/starters take an optional ?childId= , and POST /tries , /chat/messages , /caregivers , /circles/:goal/posts and /posts/:id/replies an optional childId . The app sends the child picked in its switcher (stored on the phone). Without it, lists cover every child and everything else uses the first child, so single-child families and older app builds behave as before. Daily tips take turns between children by day.

Rate limits (apps/api/src/lib/rateLimit.ts): requests count against the signed-in user, or the IP without a valid token. Every route has a ceiling of 300 a minute. Tighter limits: guest creation 20/hour, login 10 and reset 10 per 15 minutes, forgot-password 5/hour (all per IP), register 10/hour, chat messages 40/hour, moments 30/hour, pattern generation 10/hour and caregiver notes 20/hour per IP. Over the limit returns 429 with a retry-after header and a readable message.

Testing and evals

Tests (apps/api/test) cover guest/register/login auth, goal locking, moment tagging and pattern creation, the chat SSE stream, the safety bypass, feedback and bookmarks, cascade delete, and the daily job being safe to re-run. They always use the mock LLM.

Evals (`evals/cases.json` , 25 cases) measure retrieval hit rate, grounding, clarify behavior and safety-trigger precision/recall. Each run writes a JSON report to `evals/results/` . Latest mock-mode run:

Metric	Result
Expected goal in top 3	15/15
Expected chunk in top 3	12/14
Clarify decisions correct	19/19
Safety recall	6/6
Safety false positives	0/19

Two caveats on these numbers. Grounding scores are close to meaningless in mock mode, because the mock answer is stitched together from the chunk text. And one safety case was fixed after the eval caught it, so the safety score is optimistic (see `AI_MISTAKES.md` #4). A clean real-provider grounding run is still to do.

Content

Seven goals exist. Three have full content: **handling tantrums**, **fixing sleep issues** and **tackling picky eating**. The other four (focus, keeping busy, screen time, potty training) are shown as "Coming soon". Every win cites sources from `content/sources.json` (AAP/HealthyChildren.org, CDC, NHS, AASM), with access dates. The advisors shown in the app are fictional placeholders.

Privacy by design

- Only the child's birth **month and year** are stored, never the full date.
- `llm_calls` stores metrics only, never prompts or outputs. `safety_events` stores the category only.
- Notifications are scheduled on the phone, so their text (which includes the child's nickname) never passes through a push service. Notification settings and pending check-ins are stored only on the device.
- Family Playbook links are 32-byte random tokens that can be revoked. The page shows only the shared wins, the child's nickname and the parent's first name. Caregiver notes go through the same safety gate as the parent's.
- Circles show a per-circle pseudonym and a coarse label ("Mom of a 2-year-old"), never a name, email or child's nickname. The API never returns another user's id.
- Every user-owned table cascades from `users` , so deleting an account removes all of that user's data. A test checks this.

Known limitations

- **Safety is rule-based.** Recall depends on phrase lists, so phrasing we didn't anticipate can slip through. The system prompt is a second layer, not a guarantee.

- **Lexical retrieval.** Postgres FTS suits a corpus this small but misses synonyms and can match on unrelated shared words. pgvector is the planned upgrade.
- **Content needs expert review.** The AASM sleep-duration ranges in particular have not been machine-verified against the paper.
- **Payments and referrals are stubs.** The payroll writes a fake subscription row, which does unlock content.
- **Notifications are on-device only, and not available in Android Expo Go.** Testing them on Android needs a development build. They pause if the app isn't opened for about 3 days, and there's no measurement yet of how often a check-in leads to a logged moment.
- **Single instance.** The daily tip cron runs in-process.
- **Circles moderation has no admin UI yet.** The review queue is a JSON endpoint. Before launch, someone has to own that queue every day. There are no push alerts for replies, only the inbox.
- **Rate limits are in memory,** so they reset when the API restarts and aren't shared between instances. More than one instance needs the plugin's Redis store.
- One running conversation per user. No social sign-in, and no way to change email or password from Settings (password reset works). Reset emails need a Resend key; without one, codes only appear in the API log.

More docs

- [PLAN.md](#) : the original MVP plan, scope cuts and data model
- [AI_MISTAKES.md](#) : where AI-generated output was wrong, how it was caught and what changed
- [content/README.md](#) : content format and editorial rules